Contents

Git — Deep Dive Notes **1**
1. Git Internals (Object Model) . . . 1
   Object Types . . . 1
   References . . . 1
2. The Three Trees . . . 2
3. Branching Strategies . . . 2
   Trunk-Based Development (TBD) — Recommended for CI/CD . . . 2
   Git Flow — For Versioned Releases . . . 2
   GitHub Flow — Simple Web Apps . . . 2
4. Essential Commands — Advanced . . . 2
   Rebase . . . 2
   Cherry-pick . . . 2
   Bisect — Find Bug Introduction . . . 3
   Reflog — Recovery Tool . . . 3
   Stash . . . 3
5. Merge Strategies . . . 3
   Conflict Resolution . . . 3
6. Git Hooks . . . 4
   Husky Setup . . . 4
7. Signed Commits & GPG . . . 4
8. Monorepo Patterns . . . 4
   Tools . . . 4
   Git Sparse Checkout . . . 5
9. GitHub Actions Key Concepts . . . 5
10. Git Best Practices . . . 6
   Conventional Commits . . . 6
   Commit Message Rules . . . 6
 Resources . . . 6

Git — Deep Dive Notes

1. Git Internals (Object Model)

Git stores everything as objects in `.git/objects/`:

Object Types

```
blob      → file content (hash of content, not filename)
tree      → directory listing (maps names to blobs/trees)
commit    → snapshot pointer + metadata
tag       → annotated reference
```

```bash
# Inspect objects
git cat-file -t HEAD                # type
git cat-file -p HEAD                # content
git cat-file -p HEAD:src/app.py     # file at commit

# See what a tree looks like
git ls-tree HEAD
git ls-tree -r HEAD                 # recursive

# Low-level hashing
echo "hello" | git hash-object --stdin
```

References

```
.git/
```

```
├── HEAD            → points to current branch (ref: refs/heads/main)
├── ORIG_HEAD       → previous HEAD (before merge/rebase)
├── MERGE_HEAD      → commit being merged
├── refs/
│   ├── heads/      → branches
│   ├── remotes/    → remote-tracking branches
│   └── tags/       → tags
└── packed-refs     → packed references for performance
```

2. The Three Trees

```
Working Directory → git add → Staging (Index) → git commit → Repository
     HEAD~1  ← git reset HEAD~1   ←   Repository
```

- **Working Directory:** Files on disk (unstaged changes)
- **Index/Staging Area:** Snapshot prepared for next commit
- **HEAD/Repository:** Last commit snapshot

3. Branching Strategies

Trunk-Based Development (TBD) — Recommended for CI/CD

```
main (trunk)
├── Short-lived feature branches (< 2 days)
├── Merge to main frequently
├── Feature flags for incomplete features
└── No long-lived branches
```

Pros: Reduces merge conflicts, enables true CI, fast feedback **Used by:** Google, Facebook

Git Flow — For Versioned Releases

```
main ──────────────────────────────────
      ↑release/1.0                    ↑release/2.0
develop ──────────────────────────────────
         ↑feature/a   ↑feature/b   ↑hotfix/x
```

Pros: Clear release process, hotfix path **Cons:** Long-lived branches, complex merge history

GitHub Flow — Simple Web Apps

```
main ──── (always deployable)
     ↑feature/description-of-work
```

PRs reviewed and merged to main → immediate deploy

4. Essential Commands — Advanced

Rebase

```bash
git rebase main                    # replay commits on top of main
git rebase -i HEAD~5               # interactive — reorder, squash, edit
git rebase --onto main feature~3   # transplant last 3 commits to main
git rebase --abort                 # cancel rebase
git rebase --continue              # after resolving conflict
```

Cherry-pick

```
git cherry-pick abc1234            # apply single commit
git cherry-pick abc1234..def5678   # range (excludes abc1234)
git cherry-pick abc1234^..def5678  # inclusive range
git cherry-pick -n abc1234         # stage only, don't commit
```

Bisect — Find Bug Introduction

```
git bisect start
git bisect bad                     # current commit is broken
git bisect good v1.0               # last known good version
# Git checks out midpoint — test and mark:
git bisect good / git bisect bad
# Repeat until first bad commit found
git bisect reset                   # return to HEAD
# Automated bisect:
git bisect run ./test_script.sh    # script exits 0=good, non-zero=bad
```

Reflog — Recovery Tool

```
git reflog                         # all HEAD movements
git reflog show feature/foo        # branch-specific
git checkout HEAD@{5}              # recover deleted branch
git reset --hard HEAD@{3}          # undo accidental reset
```

Stash

```
git stash push -m "WIP: feature"   # save with message
git stash list
git stash pop                      # apply + drop
git stash apply stash@{2}          # apply without dropping
git stash branch feature-wip       # create branch from stash
git stash push --include-untracked # include new files
```

5. Merge Strategies

```
# Merge commit (default) — preserves branch history
git merge feature/foo

# Fast-forward only — keeps linear history
git merge --ff-only feature/foo

# Squash merge — all commits become one
git merge --squash feature/foo && git commit

# Rebase merge — linear history, no merge commit
git rebase main feature/foo && git checkout main && git merge feature/foo
```

Conflict Resolution

```
git status                         # see conflicted files
git diff --diff-filter=U           # show conflicted content

# Use a mergetool
git mergetool                      # opens vimdiff/meld/etc.
```

```
# Accept ours or theirs
git checkout --ours  path/file
git checkout --theirs path/file

# See all conflicts
grep -r "<<<<<<" .
```

6. Git Hooks

Stored in `.git/hooks/` (not committed) or use **Husky** for shared hooks.

```
# pre-commit — run linting/tests
#!/bin/bash
npm run lint
exit $?

# commit-msg — enforce commit message format
#!/bin/bash
MSG=$(cat "$1")
if ! echo "$MSG" | grep -qP "^(feat|fix|docs|style|refactor|test|chore)(\(.+\))?: .{1,72}"; then
    echo "Error: Commit message must follow Conventional Commits format"
    echo "Example: feat(auth): add JWT refresh token support"
    exit 1
fi

# pre-push — run full test suite
#!/bin/bash
npm test
```

Husky Setup

```
npm install --save-dev husky
npx husky init
echo "npm run lint" > .husky/pre-commit
echo "npm test" > .husky/pre-push
```

7. Signed Commits & GPG

```
gpg --gen-key
gpg --list-secret-keys --keyid-format=long
git config --global user.signingkey KEY_ID
git config --global commit.gpgsign true

git commit -S -m "signed commit"
git log --show-signature
git verify-commit HEAD
```

8. Monorepo Patterns

Tools

- **Nx** (Node/JS monorepos) — affected graph, task caching
- **Turborepo** (JS) — remote caching, pipeline
- **Bazel** (Google, polyglot) — hermetic builds
- **Pants** (Python focus)

Git Sparse Checkout

```
git clone --filter=blob:none --sparse https://repo.git
git sparse-checkout init --cone
git sparse-checkout set packages/api packages/shared
```

9. GitHub Actions Key Concepts

```yaml
name: CI Pipeline
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

env:
  REGISTRY: ghcr.io
  IMAGE_NAME: ${{ github.repository }}

jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        python-version: ['3.10', '3.11', '3.12']
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: ${{ matrix.python-version }}
          cache: 'pip'
      - run: pip install -r requirements.txt
      - run: pytest --cov=. --cov-report=xml
      - uses: codecov/codecov-action@v4

  build:
    needs: test
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write
    steps:
      - uses: actions/checkout@v4
      - uses: docker/setup-buildx-action@v3
      - uses: docker/login-action@v3
        with:
          registry: ${{ env.REGISTRY }}
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}
      - uses: docker/build-push-action@v5
        with:
          push: true
          tags: ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}:${{ github.sha }}
          cache-from: type=gha
          cache-to: type=gha,mode=max
```

10. Git Best Practices

Conventional Commits

```
<type>(<scope>): <description>

feat(auth): add OAuth2 login support
fix(api): handle null pointer in user endpoint
docs(readme): update installation instructions
chore(deps): upgrade axios to 1.6.0
refactor(db): extract connection pool logic
test(unit): add tests for payment service
perf(query): add index on orders.user_id
ci(github): add docker layer caching
```

Commit Message Rules

1. Use imperative mood: “Add feature” not “Added feature”
2. 72 char limit on subject line
3. Blank line between subject and body
4. Explain WHY, not WHAT (code shows what)
5. Reference issues: `Closes #123`, `Fixes JIRA-456`

Resources

- https://git-scm.com/book/en/v2 — Pro Git (free book)
- https://learngitbranching.js.org/ — Interactive git visualization
- https://ohshitgit.com/ — Common git mistake recovery
- https://www.conventionalcommits.org/ — Commit message standard
- https://trunkbaseddevelopment.com/ — TBD patterns